Modular Quiz Engine Using Factory Pattern

- Problem Statement:

Many basic quiz systems are not scalable and are difficult to extend or maintain.

- Target Users:

Students, educators, and institutions conducting quizzes.

System Design

- Architecture:

User → QuizEngine → Question (MCQ / TrueFalse)

- Key Classes:

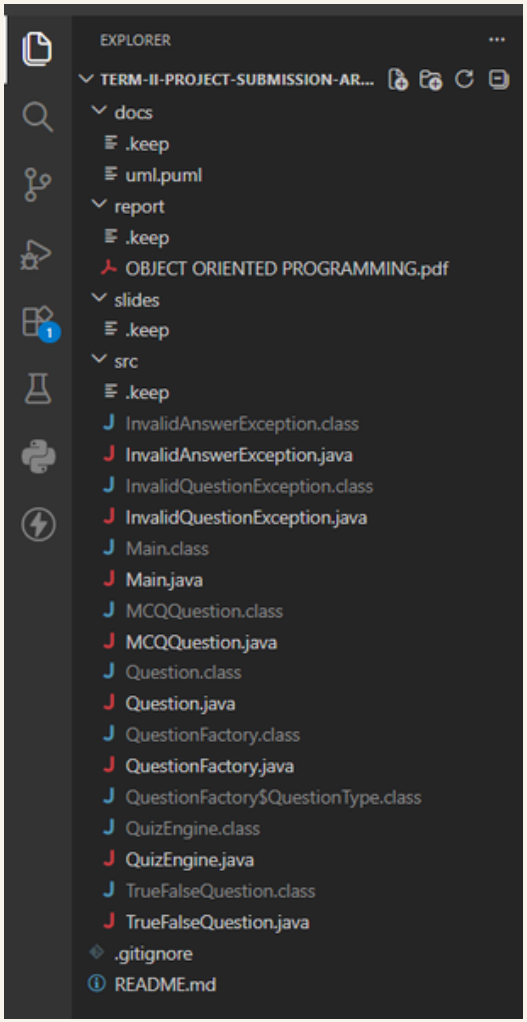
Question, MCQQuestion, TrueFalseQuestion, QuizEngine, QuestionFactory

- Packages:

model, factory, engine, exceptions, main

System Design

- Packages structure



OOP Concepts Used

- Abstraction: Question class
- Inheritance: MCQQuestion & TrueFalseQuestion extend Question
- Polymorphism: Used in QuizEngine
- Design Pattern: Factory Pattern

Technical Highlight

- Use of Factory Pattern and input validation loop.

Code Snippet — QuestionFactory.java:

```java
public class QuestionFactory {

    // Static factory method — centralised creation
    public static Question createMCQ(String text,
            List<String> options, int correctIdx, int marks) {
        return new MCQQuestion(text, options, correctIdx, marks);
    }

    public static Question createTrueFalse(
            String text, boolean correct, int marks) {
        return new TrueFalseQuestion(text, correct, marks);
    }
}

// Main.java — clean usage with no 'new MCQQuestion()' directly
engine.addQuestion(QuestionFactory.createMCQ(
    "Which keyword creates a subclass?",
    Arrays.asList("implements","extends","inherits","super"), 1, 2));
```

- Benefits:

Separates object creation, improves flexibility, and ensures safe input handling.

Live Demo

- Run quiz in terminal.
- Correct answers increase score.
- Wrong answers show correct response.
- Invalid inputs are handled safely.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\term-ii-project-submission-arushidas17> cd src
PS C:\term-ii-project-submission-arushidas17\src> javac *.java
PS C:\term-ii-project-submission-arushidas17\src> java Main

==============================
    Java OOP Knowledge Quiz
==============================
  Total Questions : 10
  Total Marks     : 14
  Type your answer and press Enter.
-----------------------------------------------

Q1. [2 marks]
  Which keyword is used to create a subclass in Java?
    A) implements
    B) extends
    C) inherits
    D) super
  Your answer (A/B/C/D...): A
  ?  Wrong. Correct answer: B) extends

Q2. [2 marks]
  Which OOP concept hides internal implementation details?
    A) Inheritance
    B) Polymorphism
    C) Encapsulation
    D) Abstraction
  Your answer (A/B/C/D...): c
  ?  Correct! +2

Q3. [2 marks]
  What does the 'final' keyword do to a method?
    A) Makes it static
    B) Prevents overriding
    C) Makes it abstract
    D) Speeds up execution
  Your answer (A/B/C/D...): 3
  [!] Please enter a letter between A and D. Please try again:
q
  [!] Please enter a letter between A and D. Please try again:
A
  ?  Wrong. Correct answer: B) Prevents overriding

Q4. [1 mark]
  Which collection allows duplicate elements in Java?
    A) HashSet
    B) TreeSet
    C) ArrayList
    D) LinkedHashSet
  Your answer (A/B/C/D...):
```

Testing & Error Handling

- Test Cases:MCQ (correct/incorrect answers)

True/False inputs (T/F, A/B, case variations)

Score calculation and grade generation

- Edge Cases:Empty input or blank answers

Invalid options (e.g., outside A–D range)

Case-insensitive inputs (e.g., “true”, “TRUE”)

Validation & Handling:Input validated before processing

Custom exceptions used for invalid inputs

User is prompted again instead of program crashing

Failure Scenarios

- Common Issues:

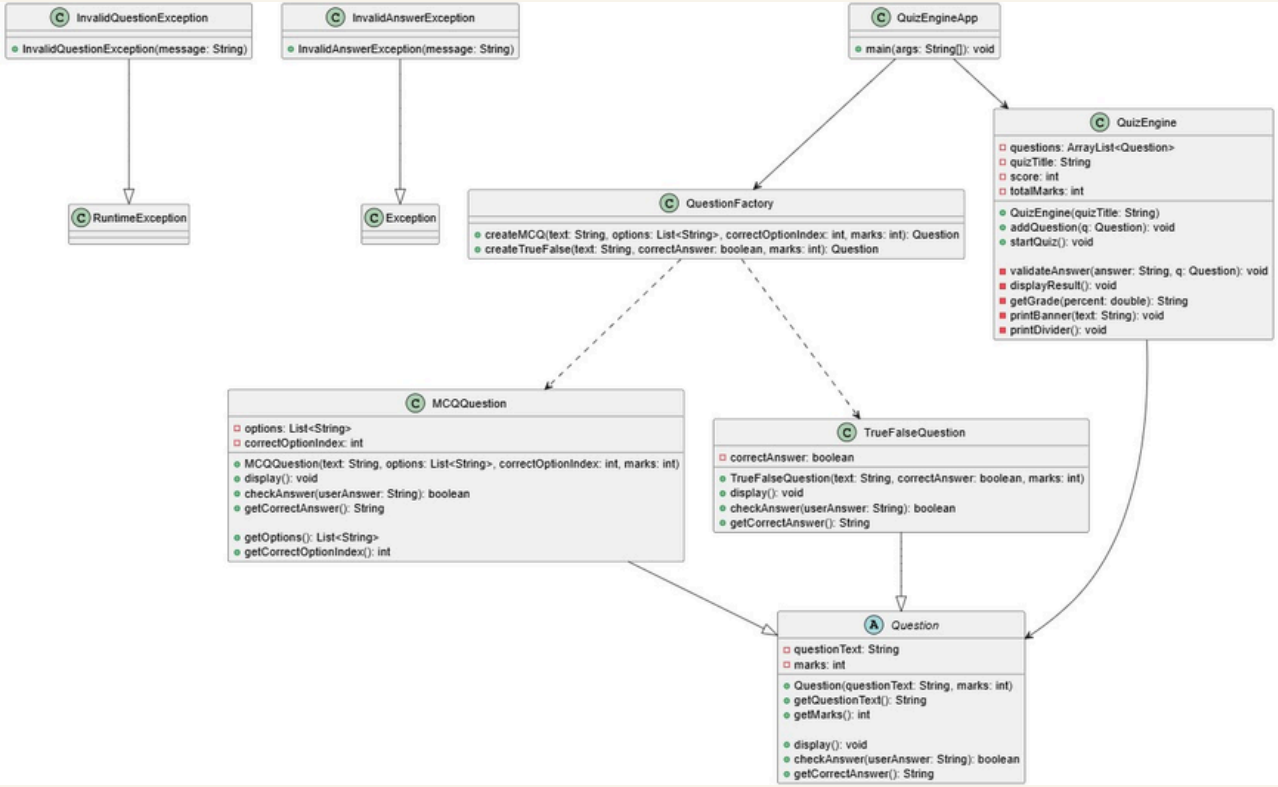
Invalid input, blank answers, wrong formats
Input outside allowed range (e.g., not A–D)
Case mismatches (e.g., TRUE, true, t)

- Handling Mechanism:

Custom exceptions (InvalidAnswerException, InvalidQuestionException)
Input validation before processing
Re-prompt mechanism to ensure valid input

Development Progression

- Step-by-step development:
- Setup → Exceptions → Classes → Factory → Engine → Main.



Conclusion

- Demonstrates OOP concepts.
- Modular and scalable design.
- Robust error handling.

Future Scope

- Add GUI, database, more question types.
- Include timer and analytics features.

THANK YOU